

Grandi Modelli di Linguaggio

Un'introduzione

Giorgio Gambosi

Università di Roma “Tor Vergata”

Abstract. *Queste note introducono i grandi modelli di linguaggio (Large Language Models, LLM) partendo da una domanda concreta: come si costruisce un sistema che risponde a domande in modo plausibile, senza avere una comprensione esplicita del mondo? Si discutono il concetto di modello di linguaggio, il meccanismo di generazione autoregressiva, il problema della scalabilità, e il processo di addestramento tramite ottimizzazione dei parametri. Si presentano infine le principali questioni aperte che definiscono la frontiera della ricerca attuale.*

Indice

1	Che cos'è un grande modello di linguaggio?	2
2	Il problema della comprensione	2
2.1	Un cambio di obiettivo	3
3	Il modello di linguaggio	4
3.1	Generazione autoregressiva	5
3.2	Il ruolo del contesto	6
4	Il problema della scalabilità	6
4.1	La soluzione: un algoritmo parametrico	7
5	I parametri del modello	7
6	L'addestramento	9
6.1	L'obiettivo di addestramento	9
6.2	La discesa del gradiente	10
6.3	I dati di addestramento	10
7	Architettura: il Transformer	11
7.1	Dalla parola al vettore: gli embedding	11
7.2	La posizione nella sequenza: codifica posizionale	13
7.3	Il meccanismo di attenzione	13
7.3.1	L'idea di base: rilevanza contestuale	13
7.3.2	Come si calcolano i pesi di attenzione	14
7.3.3	Attenzione multi-testa	16
7.3.4	La rete feed-forward e la normalizzazione	17
7.4	Encoder e decoder: le due metà del Transformer	17
7.4.1	L'encoder: costruire una rappresentazione del testo	17
7.4.2	Il decoder: generare testo in modo autoregressivo	19
7.5	Varianti architetturali: encoder-only, decoder-only, encoder-decoder	19
7.6	Layer e profondità	20
8	Oltre il pre-addestramento: il fine-tuning	20
9	Questioni aperte	21
10	Riassunto	22

1. Che cos'è un grande modello di linguaggio?

Un grande modello di linguaggio è un sistema informatico che prende in input una sequenza di testo e produce in output una risposta testuale articolata, come farebbe un interlocutore umano.



Figura 1: Un LLM riceve una domanda e produce una risposta articolata, come un interlocutore umano. La difficoltà non sta nell'interfaccia, ma nel meccanismo interno che genera la risposta.

Questa descrizione cattura il comportamento esterno, ma non dice nulla sul meccanismo interno. Capire quel meccanismo è l'obiettivo di questo testo.

Il punto di partenza è una domanda semplice: *cosa vuol dire che una risposta è corretta?* La correttezza ha due dimensioni distinte.

- **Correttezza semantica.** La risposta comunica informazioni accurate e pertinenti rispetto alla domanda posta.
- **Correttezza sintattica.** La risposta è grammaticalmente corretta nella lingua usata.

Un sistema che soddisfa entrambe le condizioni si comporta come un esperto umano: capisce la domanda, ragiona su di essa, e produce una risposta ben formata. La difficoltà è che costruire un tale sistema in modo diretto è, allo stato attuale, fuori portata.

2. Il problema della comprensione

Un sistema davvero intelligente, nel senso classico del termine, richiederebbe tre capacità fondamentali.

Prima capacità: un modello del mondo. Il sistema deve sapere cosa sono le cose. Sa cos'è uno studente, cos'è una lezione, cos'è un'università. Questa conoscenza deve essere precisa ed esaustiva: senza una rappresentazione accurata degli enti e delle relazioni tra di loro, qualunque ragionamento parte da premesse errate.

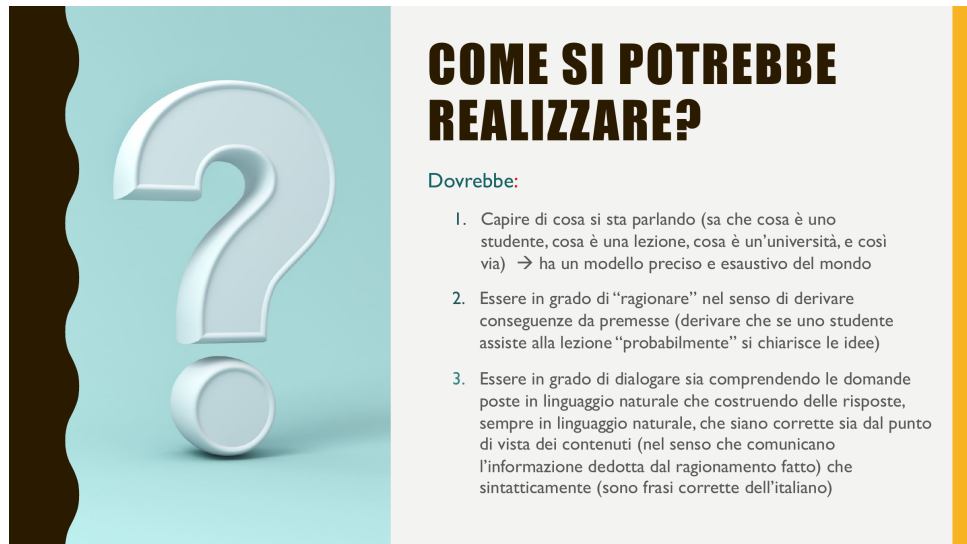


Figura 2: Le tre componenti di un sistema di dialogo ideale: conoscenza del mondo, capacità di ragionamento, e interfaccia in linguaggio naturale.

Seconda capacità: ragionamento. Il sistema deve derivare conseguenze da premesse. Se sa che uno studente frequenta le lezioni, può inferire che probabilmente capisce meglio il corso. Questo tipo di inferenza richiede meccanismi logici o probabilistici ben definiti, applicati a una base di conoscenza strutturata.

Terza capacità: linguaggio naturale. Il sistema deve comprendere domande espresse in italiano (o in qualunque altra lingua) e costruire risposte grammaticalmente corrette che comunicano il risultato del ragionamento.

Queste tre componenti corrispondono a decenni di ricerca in intelligenza artificiale simbolica. Il problema è che integrarle in un unico sistema funzionante, a scala reale, si è rivelato enormemente più difficile del previsto. Non esiste ancora un approccio che risolve questo problema in modo soddisfacente.

2.1. Un cambio di obiettivo

Di fronte a questa difficoltà, la ricerca ha adottato una strategia diversa. Invece di costruire un sistema che *capisce davvero*, si costruisce un sistema che, *visto dall'esterno*, si comporta come se capisse.

Questa idea è riassunta in un vecchio adagio anglosassone: *if it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck*. Tradotto in termini tecnici: se il sistema produce risposte che sembrano corrette a chi le legge, questo è sufficiente per molte applicazioni pratiche, indipendentemente da come funziona internamente.

Il nuovo obiettivo è quindi produrre risposte *plausibili*, non necessariamente *corrette*. La distinzione è sottile ma importante.

Definizione 2.1— Risposta plausibile vs. risposta corretta

Una risposta è **corretta** se riflette una conoscenza vera e un ragionamento valido. Una risposta è **plausibile** se è linguisticamente ben formata e coerente con ciò che di solito si scrive in risposta a domande simili. La plausibilità è

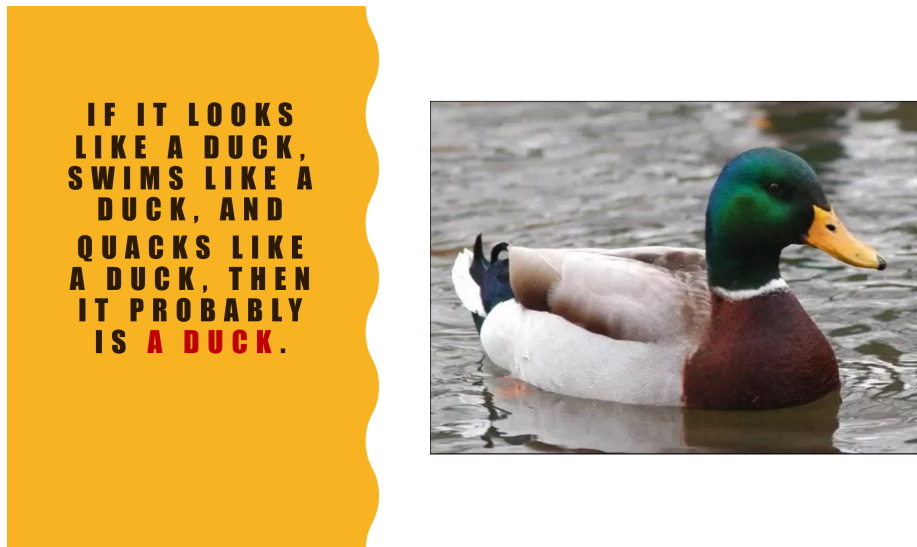


Figura 3: Il “duck test”: se cammina come un’anatra, nuota come un’anatra e starnazza come un’anatra, allora probabilmente è un’anatra. Applicato ai LLM: se risponde come un esperto, forse è sufficiente.

una condizione necessaria ma non sufficiente per la correttezza.

Osservazione 2.1— Implicazioni pratiche

Questo cambio di obiettivo ha conseguenze importanti. Un sistema orientato alla plausibilità può produrre risposte formalmente corrette ma fattualmente errate, un fenomeno che nella letteratura tecnica prende il nome di *allucinazione*. Riconoscere questa limitazione è il primo passo per usare i LLM in modo critico.

Come si costruisce allora un sistema che produce risposte plausibili? La risposta sta nell’osservare come scrivono gli esseri umani, su larga scala. Internet contiene miliardi di documenti, testi, libri, conversazioni. Questi testi mostrano, per ogni tipo di domanda, che cosa si scrive di solito come risposta. Un sistema che impara da questa enorme quantità di testo impara anche a produrre risposte che assomigliano a quelle umane.

3. Il modello di linguaggio

Il meccanismo alla base dei LLM è il **modello di linguaggio**: un sistema che, data una sequenza di parole (il contesto), calcola una distribuzione di probabilità sulla parola successiva.

L’immagine dell’urna è utile. Immaginiamo un’urna che contiene palline di colori diversi, uno per ogni parola del vocabolario. Ogni pallina ha un peso che rappresenta la probabilità di quella parola come prossima nel contesto dato. Il sistema estrae una pallina a caso, pesando le probabilità, e quella parola diventa la prossima parola nella risposta.

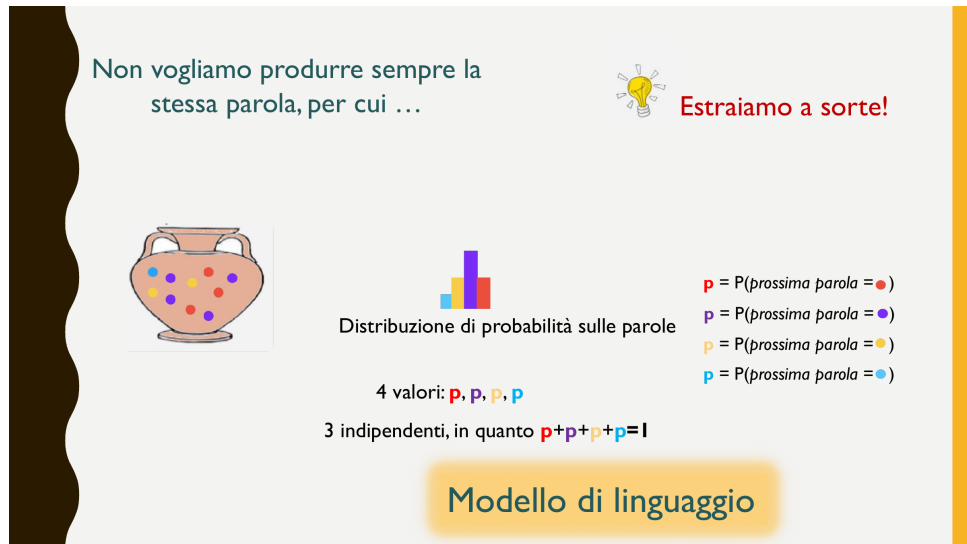


Figura 4: Il modello di linguaggio come urna: ogni parola del vocabolario ha una probabilità di essere la prossima parola, condizionata al contesto. La parola si estrae a sorte, pesando le probabilità calcolate.

Definizione 3.1— Modello di linguaggio

Un **modello di linguaggio** è una funzione che, data una sequenza di parole $w_1, w_2, \dots, w_k$, calcola la distribuzione di probabilità sulla parola successiva:

$$P(w_{k+1} \mid w_1, w_2, \dots, w_k).$$

Questa distribuzione assegna a ogni parola del vocabolario la probabilità che sia la prossima parola nella sequenza.

L'obiettivo non è produrre sempre la parola più probabile. Questo produrrebbe risposte sempre identiche per lo stesso contesto, e porterebbe spesso a testi privi di varietà e poco naturali. Si estrae invece a sorte, pesando le probabilità: le parole ad alta probabilità escono più spesso, ma c'è sempre spazio per scelte meno attese.

3.1. Generazione autoregressiva

La generazione di una risposta avviene una parola alla volta, in modo sequenziale. Questo meccanismo prende il nome di **generazione autoregressiva**.

Il processo funziona così:

1. Si parte dalla sequenza delle parole della domanda (il contesto iniziale).
2. Si calcola la distribuzione di probabilità sulla prima parola della risposta.
3. Si estrae a sorte la prima parola.
4. Si aggiunge la parola estratta al contesto.
5. Si ripete dal punto 2, usando il contesto aggiornato.

La risposta cresce così una parola alla volta. Ogni parola dipende da tutte le parole precedenti, incluse quelle della domanda originale. Il processo termina quando il modello genera un token speciale che segnala la fine della risposta.

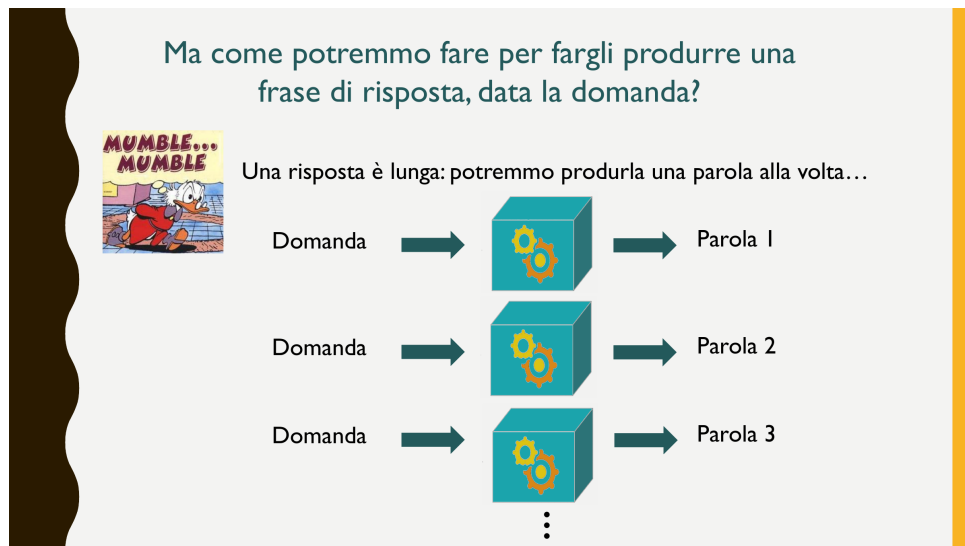


Figura 5: Generazione autoregressiva: ogni passo produce una parola, che viene aggiunta al contesto per il passo successivo. La risposta cresce progressivamente, una parola alla volta.

Esempio 3.1 — Generazione passo per passo

Supponiamo che il contesto sia “La capitale d’Italia è”. Il modello calcola le probabilità per tutte le parole del vocabolario come prossima parola. La parola “Roma” riceve probabilità alta. Si estrae “Roma”. Il contesto diventa “La capitale d’Italia è Roma”. Il modello calcola le probabilità per la parola successiva, trovando alta probabilità per il punto finale o per congiunzioni come “ed”. E così via.

3.2. Il ruolo del contesto

Un modello di linguaggio senza contesto è inutile per generare testi coerenti. Se la prossima parola dipende solo da quella precedente, il testo prodotto perde coerenza in poche parole: ogni scelta è localmente ragionevole, ma l’insieme non produce senso a lungo raggio.

Per generare testi coerenti servono contesti lunghi. Nei LLM moderni il contesto può contenere decine di migliaia di parole (il cosiddetto *context window*). GPT-4, per esempio, lavora con contesti di 128.000 token. Questo permette al modello di mantenere coerenza su interi documenti o conversazioni lunghe.

4. Il problema della scalabilità

Il modello di linguaggio, visto come una tabella di probabilità condizionali, ha un problema di dimensioni.

Consideriamo il caso semplificato: vocabolario di soli 4 simboli, contesto di 12 parole. Il numero di contesti distinti è $4^{12} = 16.777.216$. Per ognuno di questi contesti serve una distribuzione su 4 parole, cioè 3 valori indipendenti (il quarto si ricava per complemento a 1). Il totale è $3 \times 4^{12} \approx 50$ milioni di valori.

Nella realtà le proporzioni sono molto più grandi.

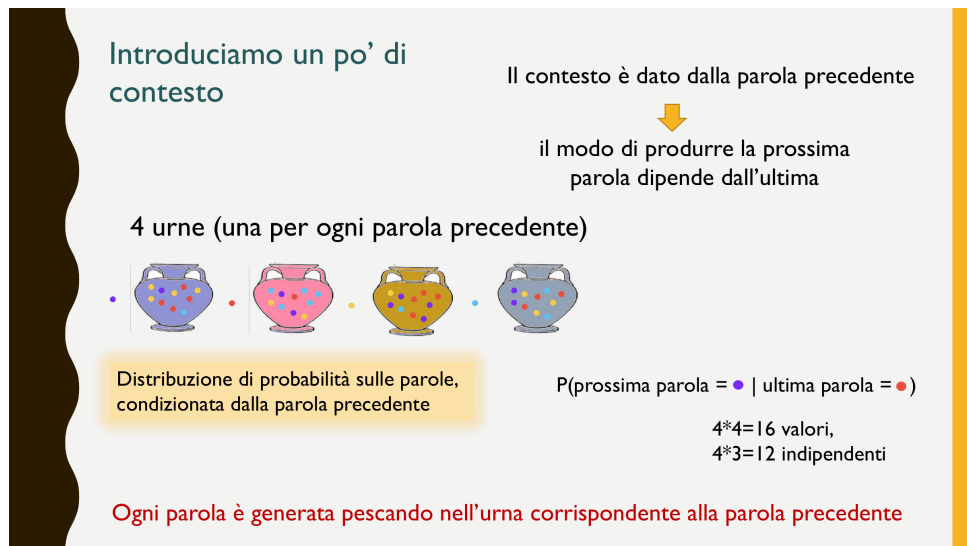


Figura 6: Estendere il contesto: la distribuzione sulla prossima parola si condiziona sulle ultime k parole. Più lungo è il contesto, più il modello riesce a mantenere coerenza a lungo raggio nel testo generato.

Ordini di grandezza reali

In un LLM come ChatGPT:

- Il vocabolario contiene circa **50.000 parole** (token, per la precisione).
- Il contesto è lungo circa **32.000 parole**.
- Il numero di contesti possibili è 50000^{32000} , un numero con centinaia di migliaia di cifre.

Nessun sistema di archiviazione può contenere questa tabella. Non è una questione di tecnologia: è una questione di grandezze fisicamente impossibili da rappresentare.

4.1. La soluzione: un algoritmo parametrico

Il problema della tabella esponenziale si risolve sostituendo la tabella con un *algoritmo*. Invece di memorizzare la distribuzione per ogni contesto possibile, si definisce una funzione che, dato un contesto qualunque, calcola la distribuzione corrispondente.

Questo algoritmo riceve in input una sequenza di k token e produce in output un vettore di V probabilità, dove V è la dimensione del vocabolario. Per un LLM moderno, l'output è un vettore di 50.000 valori, tutti non negativi e con somma pari a 1.

L'algoritmo non memorizza nulla: calcola. Questo lo rende compatto e applicabile a qualunque contesto, anche a contesti che non sono mai apparsi nei dati di addestramento.

5. I parametri del modello

L'algoritmo non ha un comportamento fisso. Il suo comportamento dipende da un insieme di valori numerici, detti **parametri**, che specificano come trasformare il

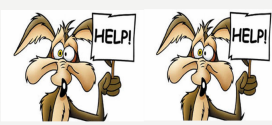
Problema 

Sono $4^{12} = 16.777.216$ urne

Non possiamo memorizzarle tutte: sono
 $3 * 16.777.216 = 50.331.648$ valori
in { , , ,  }

In realtà è molto peggio: in ChatGPT ci sono circa
50000 parole diverse (non 4) e la sequenza è lunga
32000 parole (non 12)

Come possiamo fare? Non possiamo annotarci tutte le possibilità.



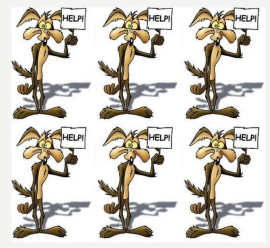


Figura 7: Il numero di contesti possibili cresce esponenzialmente con la lunghezza del contesto. Con un vocabolario reale di 50.000 parole e contesti di 32.000 parole, il numero di probabilità da stimare è astronomicamente grande.

contesto in input nella distribuzione in output.

L'immagine delle manopole è intuitiva. Ogni manopola è un valore numerico. Ruotare una manopola cambia il comportamento dell'algoritmo: per lo stesso contesto in input, l'algoritmo produce una distribuzione diversa sull'output. L'insieme di tutte le manopole determina completamente il comportamento del modello.

Definizione 5.1— Parametri di un modello

I **parametri** $\theta \in \mathbb{R}^N$ di un modello sono i valori numerici che specificano il comportamento dell'algoritmo. Il modello calcola:

$$P_{\theta}(w_{k+1} \mid w_1, \dots, w_k).$$

Parametri diversi producono distribuzioni diverse per lo stesso contesto. N è il numero di parametri, che nei LLM moderni raggiunge le decine o centinaia di miliardi.

Nei modelli reali, i parametri sono decine o centinaia di miliardi. GPT-3 ha 175 miliardi di parametri. GPT-4 ne ha, secondo le stime, più di un trilione. Questi valori definiscono la “conoscenza” che il modello ha acquisito durante l'addestramento: non in forma simbolica o dichiarativa, ma distribuita su miliardi di numeri reali.

Osservazione 5.1— Parametri e conoscenza

Un LLM non memorizza fatti come una banca dati. La conoscenza è distribuita sui parametri: si manifesta nel comportamento del modello, non in strutture dati esplicite. Questo rende difficile “leggere” cosa sa un modello, ma permette una generalizzazione flessibile a contesti nuovi.



Figura 8: Il modello visto dall'esterno: un algoritmo che riceve una sequenza di token come input e produce una distribuzione di probabilità sul prossimo token come output. L'algoritmo sostituisce la tabella esponenziale con un calcolo.

6. L'addestramento

Sapere che i parametri determinano il comportamento del modello non dice nulla su come trovare i valori giusti. Questo è il problema dell'addestramento.

6.1. L'obiettivo di addestramento

L'obiettivo è trovare i valori dei parametri che fanno sì che il modello predica bene la parola successiva su un grande corpus di testi reali. La logica è semplice: un modello che prevede bene la parola successiva in testi umani ha imparato, implicitamente, la struttura del linguaggio e del mondo che quei testi riflettono.

Da un corpus di testo si estraggono milioni di coppie (contesto, parola successiva). Per ogni coppia, il modello produce una distribuzione sulla parola successiva. Si misura quanto questa distribuzione assegna probabilità alla parola effettiva nel testo. L'obiettivo è massimizzare questa probabilità, su tutti gli esempi.

Definizione 6.1— Funzione di perdita per i LLM

Data una sequenza di testo $w_1, w_2, \dots, w_T$, la funzione di perdita standard per un modello di linguaggio è la **negative log-likelihood**:

$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log P_{\theta}(w_t \mid w_1, \dots, w_{t-1}).$$

Minimizzare $\mathcal{L}$ equivale a trovare i parametri θ che massimizzano la probabilità assegnata alle parole effettive nel corpus.



Figura 9: Dentro l’algoritmo: migliaia di parametri (metaforicamente, “manopole e pulsanti”) determinano il comportamento preciso del modello. Cambiare i parametri cambia la distribuzione prodotta per ogni contesto.

6.2. La discesa del gradiente

Trovare i parametri che minimizzano la funzione di perdita su miliardi di esempi richiede un algoritmo di ottimizzazione. L’algoritmo usato è la **discesa del gradiente stocastica** (SGD e varianti).

Il processo funziona così:

1. Si seleziona un piccolo gruppo di esempi (un *mini-batch*).
2. Si calcola la perdita su questi esempi.
3. Si calcola il gradiente della perdita rispetto a tutti i parametri: per ogni parametro, il gradiente dice in quale direzione e di quanto modificarlo per ridurre la perdita.
4. Si aggiornano tutti i parametri di un piccolo passo nella direzione opposta al gradiente.
5. Si ripete da capo su tutti gli esempi del corpus, più volte.

Il calcolo del gradiente rispetto a tutti i parametri usa la **retropropagazione** (backpropagation), un algoritmo che applica la regola della derivata composta in modo efficiente su reti con miliardi di parametri.

Osservazione 6.1— Il ruolo delle derivate

Il calcolo del gradiente si basa sulle derivate della funzione di perdita rispetto ai parametri. La derivata parziale $\partial \mathcal{L} / \partial \theta_i$ dice di quanto cambia la perdita se si modifica il parametro θ_i di una piccola quantità. Questo valore indica la direzione in cui spostare il parametro per ridurre la perdita.

6.3. I dati di addestramento

La qualità del modello dipende dalla quantità e dalla qualità dei dati. Per ChatGPT-3, il corpus di addestramento conteneva circa 570 gigabyte di testo, corrispondenti a



Figura 10: Il corpus di addestramento: ogni testo genera migliaia di coppie (contesto, parola successiva). Il modello impara a predire la parola successiva per ogni contesto osservato.

centinaia di miliardi di parole. Questi testi provenivano da pagine web, libri digitali, articoli scientifici e altri materiali scritti.

Ordini di grandezza del calcolo

Addestrare GPT-3 su un singolo computer richiederebbe centinaia di anni. Nella pratica, il calcolo si distribuisce su decine di migliaia di processori grafici (GPU) che lavorano in parallelo. L'addestramento di GPT-3 ha richiesto alcune settimane su un cluster di oltre 10.000 GPU, con un costo stimato di decine di milioni di dollari.

7. Architettura: il Transformer

L'algoritmo che calcola la distribuzione sulla prossima parola non è generico. Ha una struttura specifica, progettata per trattare sequenze testuali in modo efficiente. Questa struttura si chiama **Transformer** ed è alla base di tutti i grandi modelli di linguaggio moderni, da GPT a BERT a LLaMA. Fu introdotta nel 2017 da Vaswani et al. nel paper *Attention Is All You Need*, e in pochi anni ha sostituito quasi completamente le architetture precedenti basate su reti ricorrenti.

Per capire come funziona un Transformer conviene procedere per strati, partendo dal problema più elementare: come rappresentare le parole in una forma che un algoritmo possa elaborare.

7.1. Dalla parola al vettore: gli embedding

Le parole sono simboli discreti. "Gatto", "cane", "felino" sono tre stringhe di caratteri che un computer può confrontare solo in modo superficiale: sa se sono uguali o diverse, ma non sa che "gatto" e "felino" sono più vicini tra loro di quanto non lo siano "gatto" e "automobile". Gli algoritmi di ottimizzazione, d'altra parte, lavorano su numeri reali e si aspettano input numerici su cui calcolare derivate e gradienti.



Figura 11: La discesa del gradiente: per ogni coppia (parola prevista, parola effettiva), l'algoritmo calcola l'errore e indica come modificare ogni parametro per ridurlo. Si procede così, un passo alla volta, su tutto il corpus.

La soluzione è la **rappresentazione vettoriale** (embedding): a ogni parola del vocabolario si assegna un punto in uno spazio numerico ad alta dimensione. Ogni parola diventa un elenco di alcune centinaia o migliaia di numeri reali, uno per ciascuna dimensione di quello spazio. La posizione di quel punto non è fissata a priori: viene appresa durante l'addestramento, insieme a tutti gli altri parametri del modello.

Il risultato è che parole con significati simili tendono a occupare regioni vicine dello spazio. Non per una regola esplicita, ma perché nel corpus di addestramento compaiono in contesti simili, e il modello impara a dargli rappresentazioni simili per minimizzare l'errore di predizione. La prossimità geometrica nello spazio degli embedding riflette la prossimità semantica nel linguaggio.

Esempio 7.1— Geometria semantica negli embedding

In uno spazio di embedding ben addestrato emerge spontaneamente una struttura geometrica che rispecchia relazioni concettuali. Il punto che rappresenta “re” e il punto che rappresenta “uomo” hanno una certa differenza vettoriale. Quella stessa differenza, applicata al punto “donna”, porta in prossimità del punto “regina”. La relazione “sovrano maschile vs sovrano femminile” è codificata geometricamente, senza che nessuno l'abbia programmata: emerge come effetto collaterale dell'ottimizzazione sulla predizione della parola successiva.

Gli embedding non catturano solo il significato delle parole in isolamento. Catturano il significato *in uso*: parole polisemiche come “banco” (istituto di credito, sedile, fondale marino) potrebbero ricevere lo stesso vettore iniziale, ma all'interno del Transformer quel vettore viene continuamente aggiornato in base al contesto circostante, come vedremo nella sezione sull'attenzione.

7.2. La posizione nella sequenza: codifica posizionale

Gli embedding descrivono il significato di ogni parola, ma non dicono nulla sulla sua posizione nella frase. Eppure la posizione è informazione essenziale: la frase “il gatto insegue il topo” e la frase “il topo insegue il gatto” usano le stesse parole e quindi gli stessi embedding per ciascuna di esse. Senza informazione posizionale, il modello le tratterebbe come identiche, producendo la stessa distribuzione sulla parola successiva per entrambe. Il Transformer deve essere in grado di distinguerle.

Esempio 7.2— Perché la posizione è necessaria

Consideriamo le frasi “Mario ha visto Giulia” e “Giulia ha visto Mario”. Gli embedding di “Mario” e “Giulia” sono gli stessi in entrambe le frasi: le parole non cambiano. Senza una codifica della loro posizione, il modello non può sapere chi è il soggetto e chi è l’oggetto dell’azione. La codifica posizionale fornisce a ogni token un segnale aggiuntivo che dipende dalla sua posizione nella sequenza, rendendo la rappresentazione del primo “Mario” distinta da quella del secondo, anche se la parola è la stessa.

Per risolvere questo problema, a ogni embedding si aggiunge una **codifica posizionale**: un segnale numerico che dipende dalla posizione del token nella sequenza. Il primo token riceve un segnale, il secondo un segnale diverso, e così via. Questo segnale si combina con l’embedding della parola, producendo una rappresentazione che codifica sia il significato che la posizione.

L’addestramento impara anche questi segnali posizionali, oppure si usano funzioni matematiche fisse che coprono sequenze di qualunque lunghezza. Il risultato è che il Transformer distingue non solo *cosa* c’è in una posizione, ma anche *dove* si trova.

7.3. Il meccanismo di attenzione

Il componente centrale del Transformer è il **meccanismo di attenzione** (attention). È il meccanismo che permette a ogni parola di “guardare” le altre parole nella sequenza e di aggiornare la propria rappresentazione in base a quelle che le sono più rilevanti.

7.3.1. L’idea di base: rilevanza contestuale

Immaginiamo di dover capire il significato della parola “lui” in questa frase: “Marco è uscito presto perché lui aveva un appuntamento”. Per interpretare correttamente “lui”, dobbiamo collegarlo a “Marco”, che compare cinque posizioni prima. Questo collegamento non è né locale né automatico: dipende dal contenuto dell’intera frase.

Il meccanismo di attenzione formalizza questa operazione. Per ogni parola nella sequenza, calcola quanto ogni altra parola è rilevante per costruire la rappresentazione di quella parola. Il risultato è un insieme di pesi: numeri tra zero e uno, uno per ogni posizione nella sequenza, che indicano quanta importanza dare a ciascuna parola nel costruire la nuova rappresentazione della parola corrente. Le parole più rilevanti ricevono peso alto, le meno rilevanti peso basso o nullo.

La nuova rappresentazione di ogni parola è una combinazione delle rappresentazioni di tutte le altre, pesata da questi pesi. Una parola che riceve peso alto contribuisce molto; una che riceve peso basso contribuisce poco.

7.3.2. Come si calcolano i pesi di attenzione

Per capire il meccanismo di attenzione conviene partire da una domanda concreta. Consideriamo la frase:

Il gatto nero che viveva in giardino dormiva sul tappeto.

Quando il modello elabora la parola “dormiva”, ha bisogno di capire *chi* dorme. La risposta è “gatto”, ma “gatto” si trova sei posizioni prima, separata da tre altre parole. Come fa il modello a stabilire questo collegamento?

Il meccanismo di attenzione risolve il problema assegnando a ogni parola tre ruoli distinti, tutti derivati dalla stessa rappresentazione iniziale ma attraverso trasformazioni diverse apprese durante l’addestramento.

I tre ruoli: query, key e value

Il **primo ruolo** è la **query**: il segnale che una parola emette per cercare informazioni rilevanti nel contesto. Si può pensare alla query come a una domanda: “dormiva” emette una query che significa qualcosa come “sto cercando il mio soggetto, l’entità che compie l’azione”. “sul” emette una query diversa, che significa qualcosa come “sto cercando il luogo o l’oggetto a cui mi riferisco”. Ogni parola emette la propria query, e quella query è specifica per il ruolo sintattico e semantico che la parola ricopre nella frase.

Il **secondo ruolo** è la **key**: il segnale che una parola emette per segnalare agli altri *che tipo di informazione offre*. La key non dice cosa la parola significa in dettaglio: dice di che tipo di cosa si tratta. “gatto” emette una key che segnala qualcosa come “sono un sostantivo, un soggetto potenziale, un’entità animata”. “tappeto” emette una key che segnala “sono un sostantivo, un oggetto inanimato, un luogo potenziale”. “dormiva” stesso, nel suo ruolo di candidato per altri, emette una key che segnala “sono un verbo, un’azione”.

Il **terzo ruolo** è il **value**: il contenuto effettivo che una parola trasmette alle altre quando queste la trovano rilevante. Il value di “gatto” contiene la rappresentazione completa di quella parola in quel contesto: felino, animale domestico, entità che compie azioni, e tutto ciò che il modello ha appreso sul concetto di gatto. Il value è ciò che viene effettivamente trasferito.

La distinzione tra key e value è sottile ma fondamentale. La key serve per *decidere* se una parola è rilevante; il value è ciò che si *ottiene* da quella parola se si decide che è rilevante. Una parola può avere una key molto riconoscibile (facile da trovare) ma un value poco informativo, o viceversa. I due segnali sono indipendenti e si apprendono separatamente.

Il processo di attenzione passo per passo

Vediamo ora come funziona il processo per la parola “dormiva” nella frase di esempio.

Passo 1: generazione dei segnali. Ogni parola della sequenza genera i propri tre segnali: query, key e value. Questi tre segnali si ricavano tutti dalla stessa rappresentazione vettoriale della parola, ma attraverso tre trasformazioni diverse, apprese durante l’addestramento. La stessa parola “gatto” partecipa quindi al meccanismo di attenzione in tre modi simultaneamente: come interrogante (cosa sta

cercando), come candidato (cosa offre agli altri), e come contenuto (cosa trasferisce se viene selezionato).

Passo 2: confronto query-key. Per ogni parola che “dormiva” sta considerando come potenziale fonte di informazione, il modello confronta la query di “dormiva” con la key di quella parola. Questo confronto produce un numero: alto se i due segnali si corrispondono bene, basso se non si corrispondono. La query di “dormiva” (“sto cercando il mio soggetto”) si confronta con:

- la key di “Il”: articolo, bassa corrispondenza;
- la key di “gatto”: sostantivo animato, soggetto potenziale, alta corrispondenza;
- la key di “nero”: aggettivo, corrispondenza moderata;
- la key di “che”: pronome relativo, corrispondenza bassa;
- la key di “viveva”: verbo, corrispondenza molto bassa (non si cercano altri verbi);
- la key di “in”: preposizione, corrispondenza bassa;
- la key di “giardino”: sostantivo inanimato, luogo, corrispondenza bassa per un soggetto;
- la key di “sul”: preposizione, corrispondenza bassa;
- la key di “tappeto”: sostantivo inanimato, corrispondenza moderata ma inferiore a “gatto”.

Passo 3: normalizzazione. I valori numerici ottenuti dal confronto vengono trasformati in pesi che sommano a uno, formando una distribuzione di rilevanza sull'intera sequenza. “gatto” riceve il peso più alto, diciamo 0.55. “tappeto” riceve un peso intermedio, diciamo 0.18. Le altre parole ricevono pesi bassi che si dividono il restante 0.27. Ogni parola nella sequenza riceve un peso, anche le parole lontane o poco rilevanti: ricevono semplicemente un peso molto vicino a zero.

Passo 4: combinazione dei value. La nuova rappresentazione di “dormiva” si costruisce sommando i value di tutte le parole nella sequenza, ognuno moltiplicato per il peso che quella parola ha ricevuto. “gatto” contribuisce molto (peso 0.55): il suo value, che porta informazioni su soggetto animato, entità felina, agente dell'azione, viene incorporato in misura dominante nella nuova rappresentazione di “dormiva”. “tappeto” contribuisce in misura minore (0.18): porta informazione sul luogo dell'azione. Le parole con peso quasi zero contribuiscono in modo trascurabile.

Il risultato è che la rappresentazione di “dormiva” dopo l'attenzione non descrive più solo il concetto di dormire in astratto: descrive il dormire di un gatto, in un contesto specifico, con un soggetto e un luogo identificati. La parola ha assorbito dal contesto le informazioni necessarie per interpretarla correttamente.

Esempio 7.3— Attenzione come aggiornamento contestuale

Lo stesso processo avviene in parallelo per ogni parola della sequenza. Mentre “dormiva” cerca il suo soggetto, “nero” cerca il sostantivo che modifica (trova “gatto” con peso alto), “tappeto” cerca la preposizione che lo introduce (trova “sul” con peso alto), “viveva” cerca il suo soggetto relativo (trova “che” e poi risale a “gatto” con pesi alti).

Alla fine di un singolo layer di attenzione, tutte le parole della sequenza hanno aggiornato le proprie rappresentazioni assorbendo informazione dal contesto. Le rappresentazioni non sono più isolate: sono contestualizzate. “gatto” non è più solo il concetto di gatto: è il gatto di cui si parla in questa frase, quello

che dorme, quello che è nero, quello che viveva in giardino. Ogni parola porta in sé una traccia del significato di tutte le altre parole che le sono rilevanti.

Perché tre segnali separati e non uno solo?

Una domanda naturale è: perché usare tre segnali distinti (query, key, value) invece di confrontare direttamente le rappresentazioni delle parole?

La risposta è che i tre ruoli catturano aspetti diversi della rilevanza. La *key* descrive cosa una parola è dal punto di vista del suo ruolo come candidato: sintattico, semantico, funzionale. La *query* descrive cosa una parola *cerca*: il tipo di informazione che le manca per essere interpretata correttamente. Il *value* descrive cosa una parola *dà*: il contenuto effettivo che trasferisce quando viene selezionata. Questi tre aspetti non coincidono necessariamente. Una parola molto rilevante come candidato (alta corrispondenza tra query e key) può trasferire un contenuto relativamente limitato (value poco informativo). Una parola con un value ricchissimo può essere difficile da trovare (key che si accorda con poche query).

Separare questi tre ruoli dà al modello più flessibilità: può imparare a segnalare la propria reperibilità in modo diverso dal proprio contenuto, e a cercare informazioni in modo diverso dal tipo di informazione che offre agli altri.

Osservazione 7.1— Terminologia tecnica

Nella letteratura tecnica i tre ruoli prendono i nomi di **query** (interrogante), **key** (candidato) e **value** (contenuto). Questi termini compaiono sistematicamente in qualunque descrizione del meccanismo di attenzione. In molti paper si usa anche la notazione abbreviata Q, K, V. Conoscere questi termini permette di leggere la letteratura tecnica senza difficoltà terminologica, avendo chiaro il significato funzionale di ciascun ruolo.

Osservazione 7.2— Perché l'attenzione risolve le dipendenze a lungo raggio

I modelli precedenti al Transformer (le reti ricorrenti) leggevano la sequenza un token alla volta, da sinistra a destra, mantenendo uno “stato” che riassumeva tutto il testo letto finora. Per portare informazione dall’inizio alla fine di una sequenza lunga, quella informazione doveva sopravvivere attraverso decine o centinaia di passi intermedi, degradando progressivamente. Il meccanismo di attenzione elimina questo problema: ogni parola può confrontare la propria query con la key di qualunque altra parola nella sequenza in un singolo passo, con un collegamento diretto e privo di intermediari. La distanza nella sequenza non conta: “dormiva” può trovare “gatto” sei posizioni prima con la stessa facilità con cui troverebbe la parola adiacente, perché il confronto query-key non dipende dalla distanza ma dalla compatibilità semantica e sintattica dei segnali.

7.3.3. Attenzione multi-testa

Una singola operazione di attenzione cattura un tipo di relazione tra le parole: per esempio, la risoluzione dei pronomi. Ma in una frase si intrecciano molti tipi diversi di relazioni simultaneamente: accordo grammaticale, dipendenze semantiche, struttura

sintattica, relazioni temporali. Un singolo meccanismo di attenzione non è sufficiente a catturarle tutte.

Il Transformer usa l'**attenzione multi-testa** (multi-head attention): esegue molte operazioni di attenzione in parallelo, ognuna con i propri parametri. Ogni “testa” di attenzione si specializza su un tipo di relazione diverso. Una testa può imparare a collegare i pronomi ai loro referenti, un'altra a identificare i verbi e i loro soggetti, un'altra ancora a riconoscere sequenze di eventi temporali. I risultati di tutte le teste vengono poi combinati in una singola rappresentazione aggiornata per ogni parola.

Il numero di teste varia da modello a modello: BERT-base ne usa 12, GPT-3 ne usa 96 per layer. Ogni testa aggiunge un tipo di relazione che il modello può catturare, aumentando la capacità espressiva dell'architettura.

7.3.4. La rete feed-forward e la normalizzazione

Dopo l'attenzione multi-testa, ogni token passa attraverso una piccola rete neurale completamente connessa, applicata indipendentemente a ogni posizione. Questa **rete feed-forward** non guarda le altre posizioni: elabora la rappresentazione di ogni token individualmente, applicando trasformazioni non lineari che arricchiscono ulteriormente la rappresentazione.

L'intuizione è che l'attenzione si occupa delle relazioni tra parole (dimensione contestuale), mentre la rete feed-forward si occupa della trasformazione del significato di ogni parola individualmente (dimensione locale). Le due operazioni si complementano.

Intorno a ognuna di queste componenti si trovano due meccanismi di supporto. Le **connessioni residuali** sommano l'input di ogni componente al suo output: questo evita che il segnale si degradi attraverso molti layer e stabilizza l'addestramento. La **normalizzazione dei layer** (layer normalization) riscalda le rappresentazioni dopo ogni componente, evitando che i valori numerici esplodano o collassino verso zero durante l'addestramento. Questi due meccanismi sono indispensabili per addestrare reti profonde con decine o centinaia di layer.

Un singolo **layer Transformer** è quindi la combinazione di questi elementi nell'ordine: attenzione multi-testa, connessione residuale e normalizzazione, rete feed-forward, connessione residuale e normalizzazione. L'output di questo layer è una nuova sequenza di rappresentazioni, una per ogni token, arricchite rispetto all'input. Questo output diventa l'input del layer successivo.

7.4. Encoder e decoder: le due metà del Transformer

L'architettura Transformer originale è divisa in due parti distinte con ruoli complementari: l'**encoder** e il **decoder**. Ognuna delle due parti è una pila di layer Transformer sovrapposti, ma con differenze nel tipo di attenzione che ciascuno esegue e in come le due parti si collegano. La figura 12 mostra la struttura complessiva.

7.4.1. L'encoder: costruire una rappresentazione del testo

L'encoder riceve la sequenza di input (ad esempio, una frase da tradurre, un documento da riassumere, o una domanda a cui rispondere) e la trasforma in una

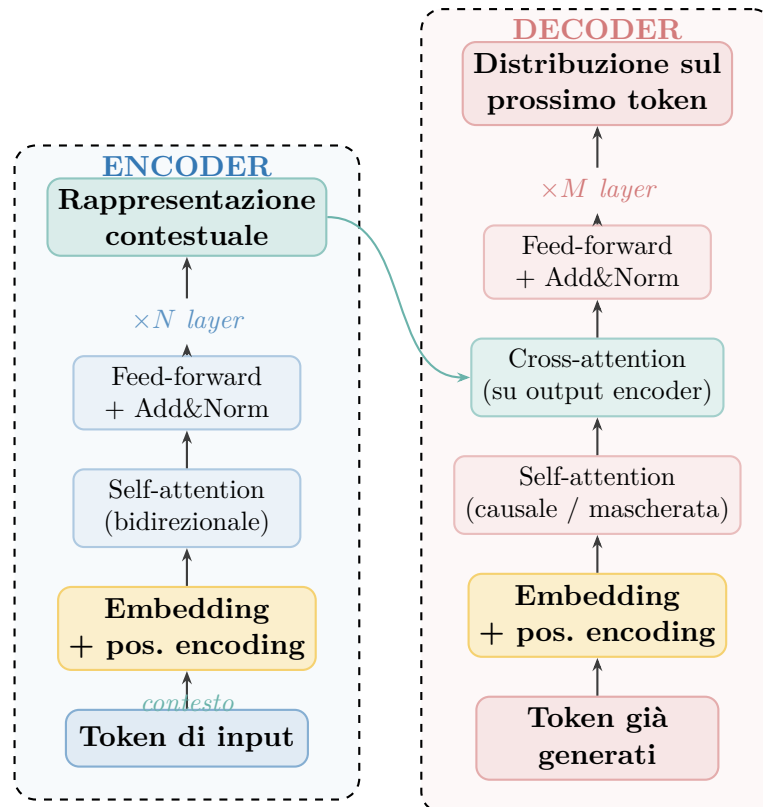


Figura 12: Architettura encoder-decoder del Transformer. L'encoder (blu) produce una rappresentazione contestuale per ogni token di input. Il decoder (rosso) genera l'output autoregressivamente, combinando self-attention causale, cross-attention sull'encoder (freccia verde) e rete feed-forward. Encoder e decoder sono pile di N e M layer rispettivamente, con N e M che possono differire.

rappresentazione contestuale: una sequenza di vettori, uno per ogni token di input, che codificano non solo il significato del token in isolamento, ma il suo significato all'interno del contesto specifico in cui compare.

L'encoder è una pila di N layer Transformer identici nella struttura, ognuno con i propri parametri. Il primo layer riceve gli embedding grezzi con la codifica posizionale. Ogni layer successivo riceve l'output del precedente. Dopo N layer, ogni token ha "visto" tutti gli altri token nella sequenza N volte, attraverso N meccanismi di attenzione successivi, e la sua rappresentazione riflette l'intero contesto in modo progressivamente più astratto.

La caratteristica fondamentale dell'encoder è che usa l'**attenzione bidirezionale**: ogni token può guardare sia i token alla sua sinistra che quelli alla sua destra. Questo è possibile perché l'encoder elabora l'intera sequenza di input in una volta, non la genera progressivamente. Può quindi sfruttare il contesto completo, sia precedente che successivo, per costruire la rappresentazione di ogni token.

Questo rende l'encoder particolarmente adatto a compiti di comprensione del testo. BERT, il modello di Google presentato nel 2018, è basato esclusivamente sull'encoder: usa l'attenzione bidirezionale sull'intera sequenza per produrre rappresentazioni ricche. Oltre a migliorare i risultati su numerosi benchmark di comprensione del linguaggio, BERT ha introdotto il paradigma del **pre-training e fine-tuning**: addestrare prima un modello generico su grandi quantità di testo non etichettato, e

poi adattarlo (fine-tuning) a un compito specifico con pochi dati etichettati. Questo paradigma ha ridefinito l’approccio allo sviluppo di modelli linguistici e rimane alla base della maggior parte dei sistemi attuali.

7.4.2. Il decoder: generare testo in modo autoregressivo

Il decoder ha il compito di generare l’output un token alla volta, in modo autoregressivo. A ogni passo, riceve due input: la sequenza di token già generati in precedenza e la rappresentazione prodotta dall’encoder per l’intera sequenza di input. Li combina per decidere quale token generare come prossimo.

Il decoder è una pila di M layer Transformer, dove M può differire da N (il numero di layer dell’encoder) in base alle scelte progettuali del modello. Ogni layer del decoder contiene tre componenti invece di due.

Il primo componente è la **self-attention causale** (o mascherata): un meccanismo di attenzione che opera sui token già generati dal decoder, ma con un vincolo fondamentale. Ogni token può guardare solo i token generati prima di lui, non quelli generati dopo. Questo vincolo è detto *causale* o *mascherato*: i token futuri vengono “mascherati” e non sono visibili. Senza questo vincolo, il decoder potrebbe “barare” durante l’addestramento, guardando il token che dovrebbe prevedere per costruire la previsione. Il vincolo causale garantisce che la generazione sia davvero autoregressiva.

Il secondo componente è la **cross-attention**: il meccanismo che collega il decoder all’encoder. Per ogni token che il decoder sta generando, la cross-attention calcola quanto ogni token nella sequenza di input dell’encoder è rilevante per questo passo della generazione. Se il decoder sta generando una traduzione, la cross-attention identifica quali parole della frase originale sono più rilevanti per la parola che si sta producendo. Se il decoder sta generando una risposta, la cross-attention identifica quali parti del testo di input sono più pertinenti alla parola corrente dell’output.

Il terzo componente è la rete feed-forward, identica a quella dell’encoder.

Dopo tutti i layer del decoder, la rappresentazione dell’ultimo token generato passa attraverso uno strato lineare finale che produce un vettore di logit, uno per ogni token del vocabolario. Quel vettore, dopo la normalizzazione tramite softmax, diventa la distribuzione di probabilità da cui si campiona il prossimo token.

7.5. Varianti architetturali: encoder-only, decoder-only, encoder-decoder

L’architettura originale del Transformer combina encoder e decoder, ma molti LLM usano solo una delle due parti. La scelta dipende dal tipo di compito per cui il modello è progettato.

Modelli encoder-only. Usano solo l’encoder con attenzione bidirezionale. Sono ottimizzati per la comprensione del testo: producono rappresentazioni contestuali ricche per ogni token, che si usano poi per classificare testi, rispondere a domande su documenti, identificare entità nominate o confrontare la similarità tra frasi. Non generano testo direttamente. BERT e le sue varianti (RoBERTa, DeBERTa, ALBERT) rientrano in questa categoria.

Modelli decoder-only. Usano solo il decoder con attenzione causale. Sono ottimizzati per la generazione autoregressiva. Non avendo un encoder separato, trattano l’intero input (domanda, contesto, istruzioni) come parte del contesto del decoder stesso: il prompt dell’utente viene semplicemente inserito come contesto iniziale nella sequenza, e il modello lo tratta come parte del “passato” da cui

condizionare il prossimo token. GPT, LLaMA, Mistral e Claude rientrano in questa categoria. È l'architettura dominante tra i modelli di conversazione.

Modelli encoder-decoder. Usano entrambe le parti. Sono ottimizzati per compiti che trasformano una sequenza in un'altra: la traduzione automatica (frase in lingua A verso frase in lingua B), il riassunto (documento lungo verso riassunto breve), la risposta a domande che richiedono sintesi di un documento. L'encoder costruisce una rappresentazione ricca dell'input; il decoder genera l'output condizionato su quella rappresentazione tramite cross-attention. T5, BART e mT5 rientrano in questa categoria.

7.6. Layer e profondità

Sia l'encoder che il decoder sono pile di layer Transformer. La profondità di questa pila è una delle variabili di progetto più importanti: più layer permettono al modello di costruire rappresentazioni progressivamente più astratte.

I layer bassi, vicini all'input, tendono a catturare strutture locali: accordo grammaticale tra parole adiacenti, riconoscimento di entità nominate, struttura sintattica di base. I layer alti, vicini all'output, tendono a catturare strutture semantiche globali: relazioni tra concetti lontani nel testo, inferenze pragmatiche, coerenza tematica.

Questa stratificazione non è progettata esplicitamente: emerge dall'addestramento. I layer specializzano spontaneamente le proprie funzioni in base al livello di astrazione necessario per minimizzare l'errore di predizione. Studi di interpretabilità meccanistica hanno osservato questa specializzazione in modo consistente su molti modelli, anche se la comprensione di cosa fa esattamente ogni layer rimane parziale, per le ragioni discusse nella sezione sulle questioni aperte.

In GPT-3 i layer del decoder sono 96, con 96 teste di attenzione per layer e $N = M = 96$. Ogni layer contiene circa 1,8 miliardi di parametri. Il totale è 175 miliardi. Questo numero non è arbitrario: più parametri e più layer permettono al modello di catturare strutture più fini, a patto di avere abbastanza dati di addestramento per stimarli senza incorrere in overfitting.

8. Oltre il pre-addestramento: il fine-tuning

Un modello addestrato a predire la parola successiva su testi di Internet non produce automaticamente un assistente utile. Produce testo plausibile, ma non necessariamente utile, sicuro, o allineato con le intenzioni dell'utente.

Per trasformare un modello di linguaggio in un assistente, si usa una tecnica chiamata **Reinforcement Learning from Human Feedback** (RLHF).

Il processo ha tre fasi:

1. **Supervised fine-tuning.** Si raccolgono coppie (domanda, risposta ideale) scritte da umani. Si usa queste coppie per addestrare ulteriormente il modello.
2. **Reward model.** Si addestra un secondo modello a predire quale di due risposte gli esseri umani preferiscono, usando confronti raccolti da valutatori umani.
3. **Ottimizzazione con RL.** Si usa il reward model come segnale di rinforzo per addestrare il modello a produrre risposte che i valutatori umani valutano positivamente.

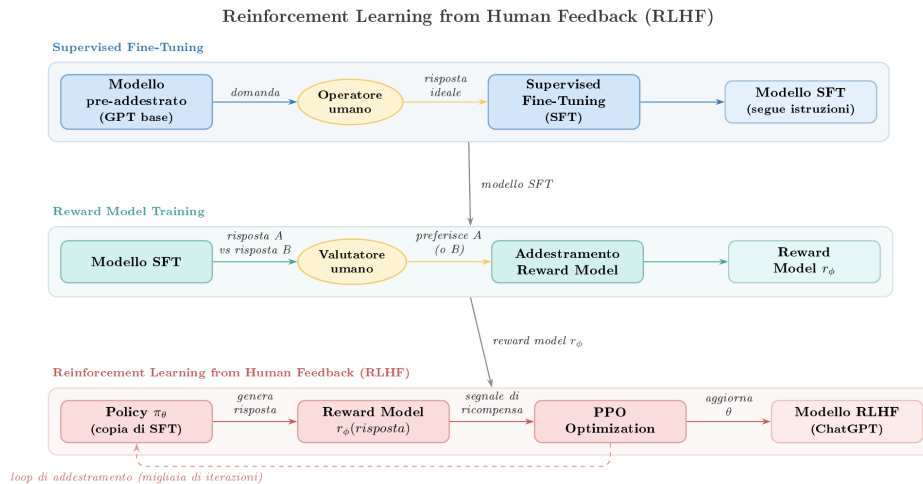


Figura 13: Il fine-tuning adatta un modello pre-addestrato a un compito specifico. Il Reinforcement Learning from Human Feedback (RLHF) usa valutatori umani per guidare il modello verso risposte più utili e sicure.

Questo processo trasforma un modello che completa testi in un modello che segue istruzioni, risponde a domande, e mantiene un comportamento sicuro.

9. Questioni aperte

I LLM sono sistemi funzionanti e utili, ma molte questioni restano aperte. Conoscerle è importante per usare questi strumenti in modo critico.

!

Attenzione, però, ci sono molte questioni aperte

MUMBLE...
Come rappresentiamo
C'è un modo più

MUMBLE...
A Gandalf servono tanti esempi da considerare
(570 GB di testo per ChatGPT).
Come eseguire tutti i passi abbas

MUMBLE...
Su un singolo computer occorrerebbero alcuni secoli...
Problema: Come far collaborare tra loro migliaia di computer?

MUMBLE...
Tutto questo deve essere implementato in un programma...
Problema: Come fare per ottenere un programma che fa esattamente quello che si vorrebbe?

MUMBLE...
HAL9000 ha tanti parametri, ma anche una struttura
Come la scegliamo?

MUMBLE...
Possiamo fare altro per migliorare
ulteriormente le risposte?

Problema: come pensare
procedimenti efficienti?

Figura 14: Le principali questioni aperte nello sviluppo e nell'uso dei LLM. Ognuna di queste questioni è un attivo fronte di ricerca.

Allucinazioni. I LLM producono testo plausibile, non testo vero. A volte generano fatti falsi con la stessa sicurezza con cui generano fatti veri. Questo è una conseguenza diretta dell'obiettivo di addestramento: il modello impara a produrre testo plausibile, non a verificare la veridicità delle proprie affermazioni.

Interpretabilità. Non si sa come i parametri del modello codificano la conoscenza. Non è possibile leggere i parametri e capire cosa il modello “sa” o “crede”. Questo rende difficile diagnosticare gli errori e correggere comportamenti indesiderati.

Ragionamento formale. I LLM mostrano capacità sorprendenti su compiti di ragionamento informale, ma falliscono sistematicamente su problemi che richiedono rigore logico o matematico. La differenza tra ragionamento apparente e ragionamento effettivo è un problema aperto.

Costi computazionali. Addestrare e usare LLM richiede enormi risorse computazionali. GPT-3 ha consumato durante l’addestramento un’energia equivalente a quella usata da circa 120 famiglie americane in un anno. Ridurre questi costi, mantenendo le prestazioni, è un problema attivo.

Sicurezza e allineamento. Un modello addestrato a massimizzare la plausibilità delle risposte può produrre contenuti dannosi, discriminatori o fuorvianti. I meccanismi di RLHF mitigano questo problema, ma non lo eliminano.

Diritti e dati. Il corpus di addestramento contiene materiale scritto da milioni di persone, spesso senza il loro consenso esplicito. Le implicazioni legali ed etiche di questo uso sono ancora oggetto di dibattito.

Messaggio chiave

Un LLM non capisce il linguaggio nel senso in cui lo capisce un essere umano. Produce testo statisticamente plausibile, addestrato su una quantità enorme di scrittura umana. Questa distinzione non lo rende inutile: lo rende uno strumento potente con limiti precisi. Conoscere questi limiti è la condizione per usarlo bene.

10. Riassunto

Questi sono i punti centrali di questa introduzione.

Un LLM produce testo plausibile calcolando, un token alla volta, la distribuzione di probabilità sul token successivo, condizionata su tutto il testo precedente.

Questa distribuzione non si memorizza in una tabella (impossibile per le dimensioni in gioco): si calcola con un algoritmo parametrico, il Transformer, i cui parametri si apprendono dai dati.

L’addestramento minimizza la probabilità che il modello assegni ai testi reali nel corpus, usando la discesa del gradiente su decine o centinaia di miliardi di parametri.

Il fine-tuning con feedback umano trasforma un modello che completa testi in un assistente che segue istruzioni.

Le capacità sono reali e spesso sorprendenti. I limiti sono altrettanto reali: allucinazioni, mancanza di interpretabilità, difficoltà con il ragionamento formale. Usare un LLM bene significa tener conto di entrambi.